

函数

Julia基础语法

苏州同元软控信息技术有限公司版权所有，
未经授权，不得复制、传播或以其他方式使用；
侵权必究。

苏州同元软控信息技术有限公司

2024年2月27日



TONGYUAN
Software and Control

为什么使用函数？

创建一个新函数让你有机会对一组语句命名，这会让你的程序更易读，也更易于调试

通过消除重复的代码，函数可以使程序更简短。而且如果以后要做修改只需要修改一处。

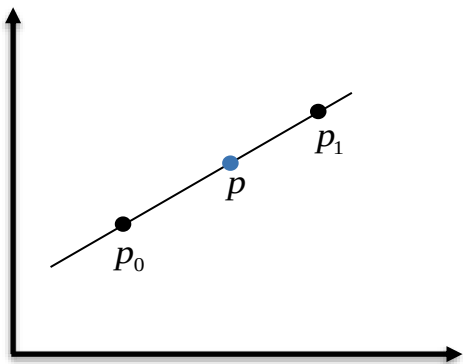
把一个长程序划分为函数，这样你就能一次调试一个部分，然后再把所有部分组装成一个整体。

设计良好的函数通常可以在很多程序中使用。一旦编写和调试一个函数以后就可以重用这个函数。

在 Julia 中，函数可以大大提高性能

Example

函数是一组一起执行一个任务的语句。



如何计算一条直线上的点？

$$y(x, x_0, y_0, x_1, y_1) = x \frac{y_1 - y_0}{x_1 - x_0} - \frac{x_1 + x_0}{2} \frac{y_1 - y_0}{x_1 - x_0} + \frac{y_1 + y_0}{2}$$



```
function LineWithProtected(x,p0,p1)
    m = (p1[2] - p0[2]) / (p1[1] - p0[1])
    b = (p1[2] + p0[2] - m * (p1[1] + p0[1])) / 2.0
    y = m * x + b
    return y
end
```

苏州同元软控信息技术有限公司
版权所有，侵权必究。



目录

1. 函数声明

2. 函数调用

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

1. 函数定义

如何定义函数？

一般结构：

方式一，完整结构：

```
function functionname(args)
    expression
    expression
    expression
    ...
    expression
end
```

方式二，简化写法：

```
functionname(args)=expression
```

```
julia> function f(x, y)
    return x + y
end
```

```
julia> function f(x, y)
    x + y #可以不写return, 与上等价
end
```

```
julia> f(2, 3) #按照默认位置传参
5
```

说明：若不写return，则函数返回值是最后计算的表达式的值

```
julia> f(a) = a * a
f (generic function with 2 methods)
```

```
julia> g = f; g(5) #函数对象可以像值一样被传递
25
```

1. 函数定义

操作符也是函数

在Julia中，大多数操作符实质是支持特殊语法的函数（&&和||等具有特殊语义的操作符除外）。

```
julia> (1+2+3)或者1+2+3  
6
```

```
julia> +(1, 2, 3) #操作符也当成函数进行调用  
6
```

```
julia> add = + #函数对象：可以像值一样被传递  
+ (generic function with 723 methods)
```

```
julia> add(1, 2, 3)  
6
```

1. 函数定义

变参函数构建

定义有任意个参数的函数会带来很多便利，这类函数通常被称为“变参”函数，即“参数数量可变”的简称。

可以通过在最后一个参数后增加省略号来定义一个变参函数。

```
julia> function bar(a, b, xs...)
    println("xs=$xs")
end
bar (generic function with 1 method)

julia> bar(1, 2)
xs=()

julia> bar(1, 2, 3)
xs=(3,)

julia> bar(1, 2, 3, 4)
xs=(3, 4)
```

递归函数构造

函数递归：函数的定义中使用函数自身的方法
例如：计算斐波那契数

```
julia> function fib(n::Int)
    if n <= 2
        y=1
    else
        y=fib(n-1)+fib(n-2)
    end
    return y
end

julia> fib(10)
55
```

目录

1. 函数声明

2. 函数调用

苏州同元软控信息技术有限公司版权所有，侵权必究。

2. 函数调用

单输入单输出

一般结构:

```
output = functionname(args)
```

```
julia> function fib(n::Int)
    if n <= 2
        y=1
    else
        y=fib(n-1)+fib(n-2)
    end
    return y
end
```

```
julia> y = fib(10)
55
```

多输入多输出

一般结构:

```
output1, output2,..., outputn
= functionname(arg1,arg2,...,argm)
```

多个形参按照默认位置传入

多个返回值按照默认位置传出

```
julia> function mul(x, y)
    return x + y, x * y
end
mul (generic function with 1 method)
```

```
julia> s,m = mul(5,10) #默认位置传入
传出
(15, 50)
```

```
julia> s
15
```

2. 函数调用

关键字参数

Julia 关键字参数允许通过名称而不是仅通过位置来识别参数，使得这些复杂函数易于使用和扩展。
使用关键字来标记参数，需要在函数的未标记参数之后使用分号，并在其后跟一个或多个关键值对 key=value 。

```
julia> function foo(a, b; c, d)
    println("a is $a")
    println("b is $b")
    return "c => $c, d => $d"
end
foo (generic function with 1 method)

julia> foo(10, 20; c = 30, d = 40) #调用时也需要使用分号
a is 10
b is 20
"c => 30, d => 40"

julia> foo("Hello", "Julia"; d = 40, c = 30) #根据形参名称传参
a is Hello
b is Julia
"c => 30, d => 40"
```

2. 函数调用

可选参数

在函数中我们可以设置参数默认值，这样在没有提供该参数的时候，就可以使用默认值来计算。

```
julia> function pos(a, b, c = 0)
    println("$a, $b, $c")
end
pos (generic function with 2 methods)

julia> pos(10, 30)
10, 30, 0

julia> pos(10, 30, 50)
10, 30, 50
```

2. 函数调用

向量化调用-广播

任何 Julia 函数 f 能够以元素方式作用于任何数组（或者其它集合），这通过语法 $f(A)$ 实现。

例如， $\sin$ 可以作用于向量 A 中的所有元素

```
julia> A = [1.0, 2.0, 3.0]
3-element Vector{Float64}:
 1.0
 2.0
 3.0

julia> sin.(A)
3-element Vector{Float64}:
 0.8414709848078965
 0.9092974268256817
 0.1411200080598672
```

2. 函数调用

向量化调用-map和匿名函数

一般结构:

```
map(fun, coll.....)
```

map(fun, coll)通过将 fun函数应用于每个元素来转换集合 coll。
对于多个集合参数，按元素调用 fun。

在 Julia 中，匿名函数可用于许多地方，例如 map()。
匿名函数的语法使用符号 " $\rightarrow$ "。

```
julia> map(sin, [1, 2, 3])  
3-element Vector{Float64}:  
 0.8414709848078965  
 0.9092974268256817  
 0.1411200080598672
```

```
julia> x -> x^2 - 2x + 1 #匿名函数声明  
#17 (generic function with 1 method)  
julia> map(x -> x^2 - 2x + 1, [1, 2, 3])  
3-element Vector{Int64}:  
 0  
 1  
 4
```

```
julia> map(+, [1, 2, 3], [10, 20, 30])  
3-element Vector{Int64}:  
 11  
 22  
 33
```

2. 函数调用

函数参数中的Do结构

把函数作为参数传递给其他函数是一种强大的技术，但它的语法并不总是很方便。

当函数参数占据多行时，这样的调用便特别难以编写。

Julia 提供了一个保留字do，用于更清楚地重写此代码：

```
julia> map(x -> begin
    if x < 0
        return -1
    elseif x == 0
        return 0
    else
        return 1
    end
end, [-2, 0, 2])
```



```
# do结构更优雅
julia> map([-2, 0, 2]) do x
    if x < 0
        return -1
    elseif x == 0
        return 0
    else
        return 1
    end
end
```

2. 函数调用

函数的复合与链式调用

Julia中的多个函数可以用函数复合或管道连接（链式调用）组合起来。

函数复合：(f ∘ g)(args...) 就等价于 f(g(args...))

```
julia> sqrt(+(3,6)) #先加后求平方根
3.0

julia> (sqrt ∘ +)(3, 6) #先加后求平方根
3.0
```

函数链式调用：把一个函数作用到前一个函数的输出上

```
julia> sqrt(sum(1:10))
7.416198487095663

julia> 1:10 |> sum |> sqrt #从左到右计算
7.416198487095663
```

说明：用 `\circ<tab>` 来输入函数复合运算符

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

请各位专家指正！

苏州同元软控信息科技有限公司版权所有，侵权必究。
传播或以其他方式使用；